

1. INTRODUCTION

- Algorithms
- Data structures
- Abstract data types
- Programming with lists and sets

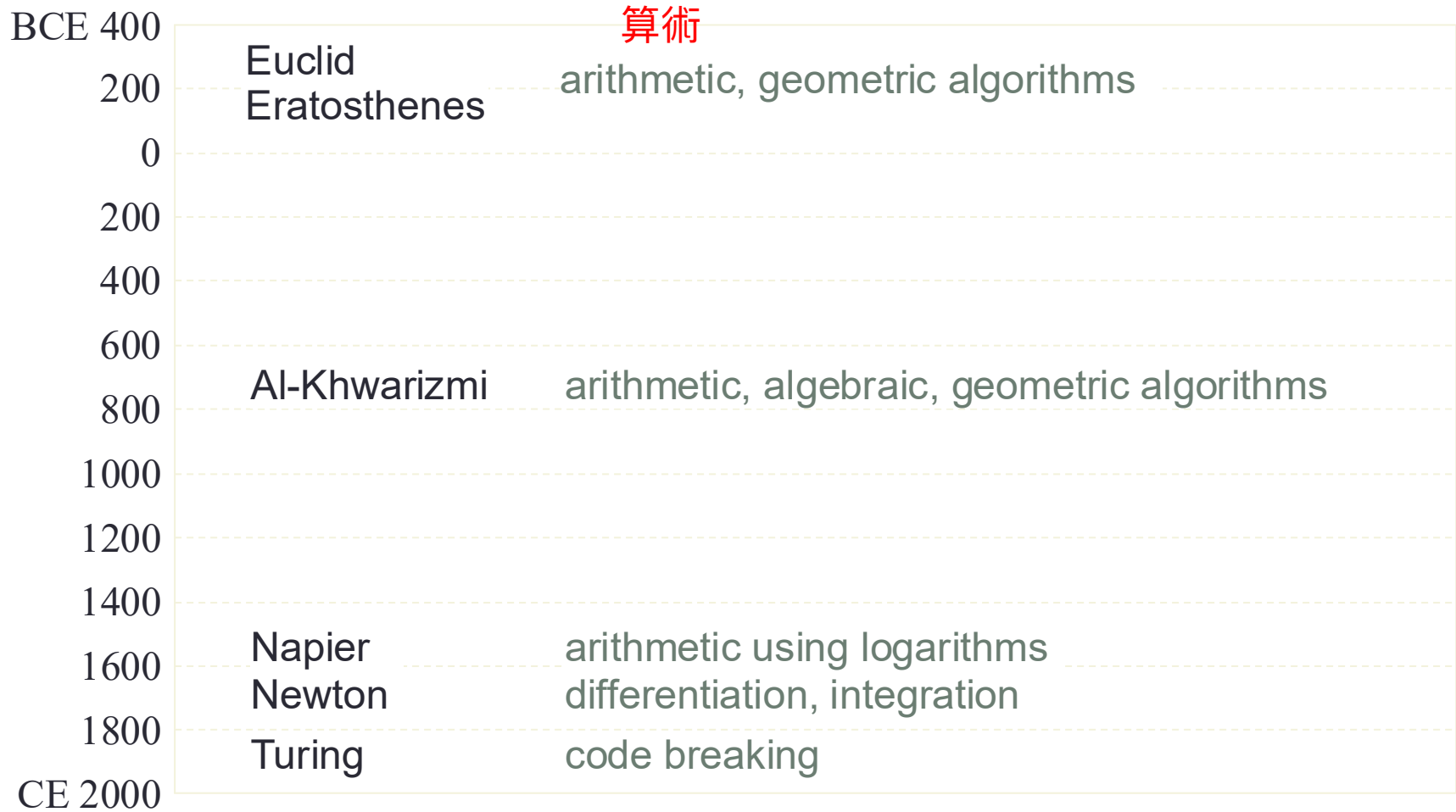
A whistle-stop tour!

- don't expect to
understand things
properly at this stage

Algorithms

- An **algorithm** is a step-by-step procedure for solving a stated problem.
- For example, consider the problem of multiplying two integers. There are many algorithms for solving this problem:
 - multiplication using a table (small integers only)
 - long multiplication
 - multiplication using logarithms
 - multiplication using a slide rule
 - binary multiplication (in computer hardware)
 - multiplication using a Fourier transformation (for huge integers)

History of algorithms



Example: finding a midpoint (1)

- **Midpoint algorithm:**

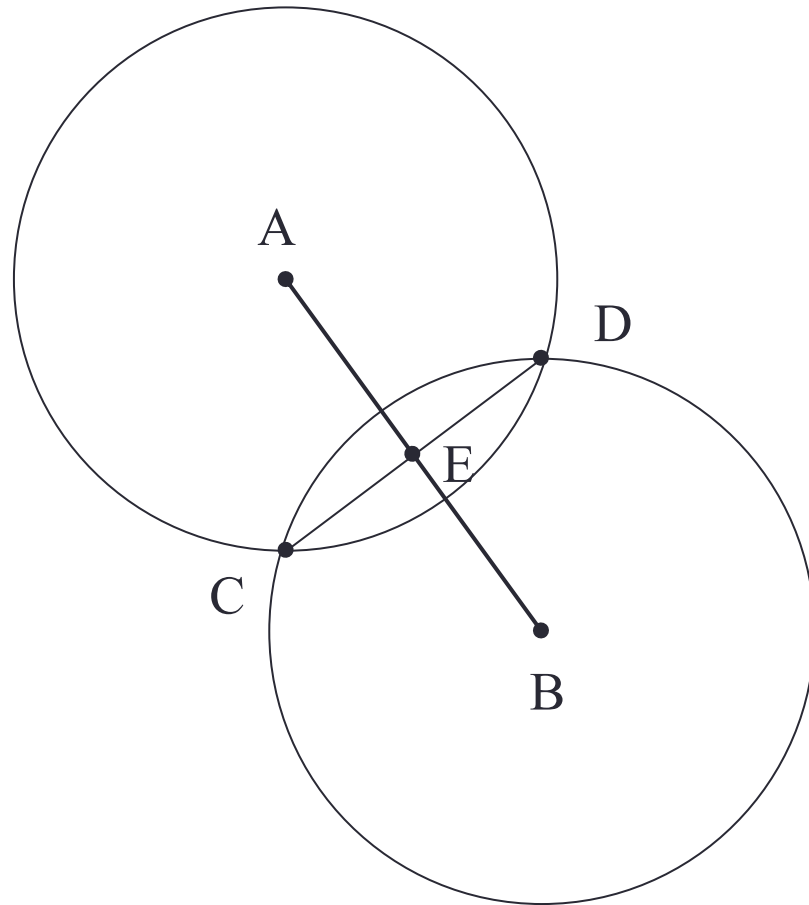
To find the midpoint of a given straight-line segment AB:

1. Draw intersecting circles of equal radius, centered at A and B respectively.
 2. Let C and D be the points where the circles intersect.
 3. Draw a straight line between C and D.
 4. Let E be the point where CD intersects AB.
 5. Terminate yielding E.
- This algorithm can be performed by a human equipped with drawing instruments.

Example: finding a midpoint (2)

- Animation:

1. Draw intersecting circles of equal radius, centered at A and B respectively.
2. Let C and D be the points where the circles intersect.
3. Draw a straight line between C and D.
4. Let E be the point where CD intersects AB.
5. **Terminate yielding E.**



Example: computing a GCD (1)

- The **greatest common divisor (GCD)** of two positive integers is the largest integer that exactly divides both. E.g., the GCD of 77 and 21 is 7.
- **Euclid's GCD algorithm:**

To compute the GCD of positive integers m and n :

1. Set p to m , and set q to n .
 2. While q does not exactly divide p , repeat:
 - 2.1. Set p to q , and set q to $(p \text{ modulo } q)$. ----- simultaneously
 3. Terminate yielding q .
- This algorithm can be performed by a human, perhaps equipped with an abacus or calculator.

Example: computing a GCD (2)

- Animation:

To compute the GCD of positive integers m and n :

1. Set p to m , and set q to n .
2. While q does not exactly divide p , repeat:
 - 2.1. Set p to q , and set q to $(p \bmod q)$.
3. **Terminate yielding q .**

m	77	n	21
p	14	q	7

Example: computing a GCD (3)

- Implementation in Java:

```
static int gcd (int m, int n) {  
    // Return the greatest common divisor of m and n  
    // (assumed positive).  
    int p = m, q = n;  
    while (p % q != 0) {  
        int r = p % q;  
        p = q;    q = r;  
    }  
    return q;  
}
```

Algorithms vs programs

- Algorithms:
 - can be performed by humans or machines
 - may be expressed in any suitable language
 - may be as abstract (undetailed) as we like.
- Programs:
 - can be performed only by machines
 - must be expressed in a programming language
 - must be detailed and specific.

Algorithm notation (1)

- In this course we express our algorithms in precisely-worded **English**.

- Steps may be numbered:

1. Do this.

2. Do that.

Do this, and then
do that.

- A conditional's extent is indicated by indentation:

3. If *proposition*: If the proposition is true,
 3.1. Do this. do this and then do that.
 3.2. Do that.

4. Else: If the proposition is
 4.1. Do the other thing. false, do the other thing.

Algorithm notation (2)

- A loop's extent is indicated by indentation:

- | | |
|--|---|
| <ol style="list-style-type: none"> 5. While <i>proposition</i>, repeat: <ol style="list-style-type: none"> 5.1. Do this. 5.2. Do that. 6. For $i = m, \dots, n$, repeat: <ol style="list-style-type: none"> 6.1. Do this. 6.2. Do that. | <p>----- As long as the proposition is true, repeatedly do this and then do that.</p> <p>----- With the variable i ranging over the values m through n, repeatedly do this and then do that.</p> |
|--|---|

- One algorithm can “call” another algorithm, or call itself recursively.
- *Note:* The numbering of steps is optional.

We won't be strict about the notation – as long as the meaning is clear (and algorithm is not written in a programming language)

Implementing algorithms

- If we wish to use the algorithm on a computer, we must first implement it in a **programming language**.
- A given algorithm could be coded in several ways, and in any suitable programming language. But all the resulting programs are implementations of the same underlying algorithm.
- In this course we express our implementations in **Java**. (Alternatives would be C, Python, etc.)

Data structures

- A **data structure** is a systematic way of organizing a **collection** of data.
- A **static data structure** is one whose capacity is **fixed** when it is first constructed.
E.g., an *array*.
- A **dynamic data structure** is one whose capacity is **variable**, so it can expand or contract at any time.
E.g., a *linked-list* or *binary-search-tree*.
- For each data structure we need **algorithms** for insertion, deletion, searching, etc.

Example: representing strings

- Possible data structures to represent the string “Java”:

Array:

0	1	2	3
'J'	'a'	'v'	'a'

Linked-list:

```
graph LR; P[ ] --> N1["'J' •"]; N1 --> N2["'a' •"]; N2 --> N3["'v' •"]; N3 --> N4["'a' •"];
```

Example: representing lists

- Possible data structures to represent the list of words «the, cat, sat, on, the, mat»:

Array:

0	1	2	3	4	5	6	7
the	cat	sat	on	the	mat		

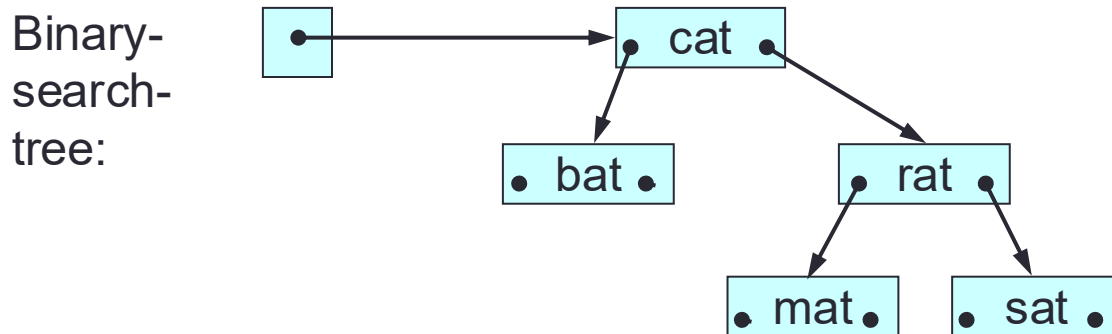
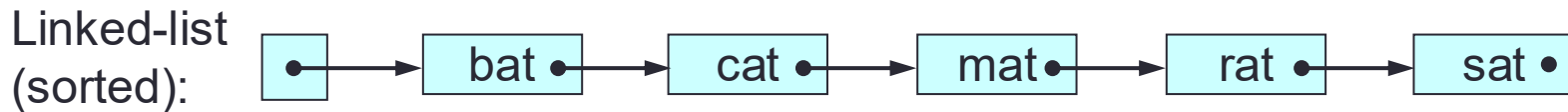
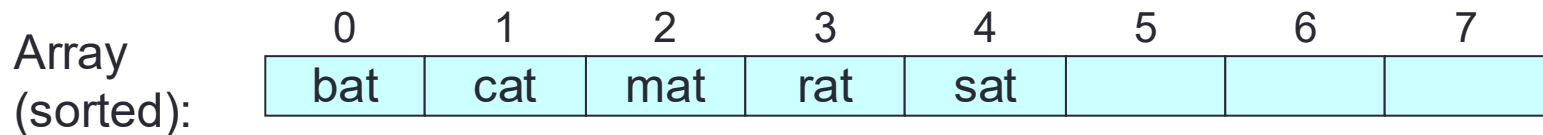
Linked-list:



Concepts like “list” and “set” will be discussed later - ADTs

Example: representing sets

- Possible data structures to represent the set of words {mat, bat, rat, cat, sat}:



Abstract data types (1)

- When we write Java application code involving strings, we don't care how strings are represented. We just declare `String` variables and manipulate them using `String` operations.
- Similarly, when we write application code involving lists, we don't care how lists are represented. We just declare `List` variables and manipulate them using `List` operations.
- Similarly, when we write application code involving sets, we don't care how sets are represented. We just declare `Set` variables and manipulate them using `Set` operations.

Abstract data types (2)

- An **abstract data type (ADT)** is a data type whose representation is **private**, and therefore of no concern to the application code.
 - E.g.: `String`, `List`, `Set`.
- ADTs are an essential technique for designing and implementing large programs.

String ADT

- A **string** is a sequence of characters.
- Useful features of the `String` ADT in `java.lang`:

```
String str; ----- declares a variable str  
str = ""; ----- that will refer to a string of  
str = "abc"; ----- characters  
str = str + "de"; ----- concatenation of strings  
int n = str.length(); ----- length of string str  
char c = str.charAt(i); ----- character at position i in  
String sub = ----- str (position 0 is leftmost)  
str.substring(i, j); ----- substring of characters at  
positions i, ..., j-1 in str
```

List ADT (1)

- A **list** is a sequence of elements, in a fixed order. (The order of the elements depends on the positions where they were inserted.)

- Useful features of the List ADT in `java.util`:

`List<Integer> ints;` declares a variable
`ints` that will refer to a
list of integers

`List<Date> dates =` declares a variable
`new ArrayList<Date>();` `dates` that will refer to a
list of dates

..... constructs an empty list
of dates (`ArrayList`
will be explained further
in §8)

List ADT (2)

- Useful features of the List ADT (*continued*):

`dates.add(d);` ----- inserts date `d` at the end of list `dates`

`dates.add(i, d);` ----- inserts date `d` at position `i` in list `dates` (position 0 is leftmost)

`dates.remove(i);` ----- removes the element at position `i` in list `dates`

if (`dates.isEmpty()`)
...
for (`Date d : dates`)
... ----- true if list `dates` contains no elements

----- iterates over list `dates`, making `d` refer to each element in turn

Example: programming with lists (1)

- Extracting a list of words from a given string (where a “word” is a sequence of consecutive letters):

```
static List<String> scan (String line) {  
    String[] wordArray =  
        line.split("[^[A-Z]&&[a-z]]+");  
    List<String> wordList =  
        new ArrayList<String>();  
    for (String word : wordArray)  
        wordList.add(word);  
    return wordList;  
}
```

Example: programming with lists (2)

- Complete program:

```
import java.util.*;
class ListingWords {
    static List<String> scan (String line) {
        ...
    }
    static void main (...) {
        List<String> words =
            scan("To be, or not to be");
        for (String w : words)
            System.out.println(w);
    }
}
```

Output:

To
be
or
not
to
be

Set ADT (1)

- A **set** is a collection of elements, in no fixed order, and in which no two elements are equal.
- Useful features of the Set ADT in `java.util`:

```
Set<Integer> ints;
Set<Date> dates =
    new TreeSet<Date> ();
```

declares a variable
`ints` that will refer to a
set of integers

declares a variable
`dates` that will refer to a
set of dates

constructs an empty set
of dates (`TreeSet` will
be explained in §10)

Set ADT (2)

- Useful features of the Set ADT (*continued*):

<code>dates.add(d);</code>	-----	adds date <code>d</code> to set <code>dates</code> (if not already in the set)
<code>dates.remove(d);</code>	-----	removes date <code>d</code> from set <code>dates</code>
<code>if (dates.isEmpty())</code> ...	-----	true if set <code>dates</code> contains no elements
<code>if (dates.contains(d))</code> ...	-----	true if date <code>d</code> is an element of set <code>dates</code>
<code>for (Date d : dates)</code> ...	-----	iterates over set <code>dates</code> , making <code>d</code> refer to each element in turn

Example: programming with sets (1)

- Program:

```
import java.util.*;
class FruitMarket {
    static void main (String[] args) {
        Set<String> basket =
            new TreeSet<String>();
        for (String a : args)
            basket.add(a);
        for (String f : basket)
            System.out.println("  " + f);
        if (! basket.contains("orange"))
            System.out.println("No orange!");
    }
}
```

Example: programming with sets (2)

- If the program's arguments are:

```
kiwi apple mango banana apple
```

the program's output will be:

```
apple  
banana  
kiwi  
mango  
No orange!
```

- The order of adding elements to a set does not matter. (The elements are in no particular order.)
- Adding an element already in a set has no effect. (No two elements of a set are equal.)